

urthreads

Engagement software for sites that do not run a server

ENGINEERING & ARCHITECTURE DOCUMENT

Version 1.0 · 8 September 2026

Repository state: `main @ 124e24a`, npm release 1.2.2

All figures independently verified against the working tree

19,605 LINES OF SOURCE	229 AUTOMATED TESTS	17 TABLES AND INDEXES	35 COMMITTS SINCE 19 MAY 2026
--	---	---	---

Contents

1. Overview

- 1.1 What it is made of
- 1.2 The design position

2. At a glance

- 2.1 Verified metrics
- 2.2 Composition

3. The runtime shape

- 3.1 The public surface
- 3.2 The admin surface

4. Security architecture

- 4.1 CORS, and the one rule that cannot bend
- 4.2 Sessions without a session server
- 4.3 CSRF on an API that is cross-origin by design
- 4.4 Rate limiting with no shared memory
- 4.5 Errors that say nothing and can still be traced
- 4.6 Email at rest

- 4.7 What the tests actually assert

5. The data model

6. Setup, and the parts that refuse

7. The dashboard

8. What this project demonstrates

9. Open problems

10. Conclusion

Appendix A: Verification log

- A.1 Tests
- A.2 Scale
- A.3 Dependencies
- A.4 Surface
- A.5 Rate-limit constants

1. Overview

urthreads gives a static website likes, threaded comments, moderation and analytics, without a server. One Cloudflare Worker, one D1 database, a static admin dashboard, and a CLI that sets the whole thing up. It is published on npm and runs on infrastructure the site owner controls.

A blog built with Jekyll, Hugo or Eleventy has no backend, which is most of why people choose one. The moment it needs comments, the usual answers are to hand the pages to a third party who then owns the audience data and puts a script tag on every page, or to abandon the static build and run a server.

urthreads is the third answer. The engagement layer is a single Worker at the edge, the data is in the owner's own D1 database, the moderation UI is three static files served from the owner's own site, and nothing in the stack has an operator other than the person who deployed it.

The project is deliberately unglamorous, and that is why it is in this portfolio. It is the one system here with no model in it, and every hard problem in it is a problem that has been hard for twenty years: authentication on a surface with no session server, CSRF on an API called cross-origin by design, rate limiting with no shared memory, and a schema that has to be right the first time because migrations on a stranger's database are not something a published package gets to do.

1.1 What it is made of

PIECE	WHAT IT IS
<code>src/worker.js</code>	The Cloudflare Worker: 20 endpoints, public and admin, 2,204 lines
<code>src/worker-security.mjs</code>	The security helpers, extracted so they can be unit tested, 544 lines
<code>src/schema.sql</code>	Eight tables, and the indexes they need
<code>web/</code>	The static admin dashboard: markup, behaviour, styles
<code>client/</code>	Two browser scripts a site embeds: <code>likes.js</code> , <code>comments.js</code>
<code>src/cli.js</code> and friends	30 CLI commands: setup, deploy, moderation, key rotation, teardown

1.2 The design position

The owner runs it, so the software has to assume the owner is not an operator. That single assumption drives most of the engineering. The guided setup can create the D1 database, write `.env` and `wrangler.toml`, initialise the schema and deploy the Worker, because a person who chose a static site did so to avoid exactly this work. The CLI refuses a wildcard CORS origin rather than warning about it. The dashboard refuses to start a session against a plain `http://` worker origin except on localhost. Defaults are the security policy, because a default is what almost every install will actually run.

The security surface is small and it is entirely on the admin side. The public endpoints take a path and return a count, or take a comment and store it as pending. The admin endpoints can delete, approve, and read everyone's data. Everything in Section 4 exists because those two sets of endpoints are served by one Worker on one origin.

2. At a glance

2.1 Verified metrics

MEASURE	VALUE
Source	19,605 lines (JavaScript, SQL, CSS, HTML)
Tests	229 passed, 0 failed, 0 skipped in 444ms
Test files	14, across the CLI, the setup helpers and the Worker
Worker	2,204 lines, 20 routed endpoints
Security module	544 lines, 25 exported functions
Database	8 tables, 9 indexes
CLI commands	30
Runtime dependencies	0
Development dependencies	1 (<code>wrangler</code>)
Commit history	35 commits, 19 May to 3 September 2026
Published	npm <code>urthreads</code> , version 1.2.2, MIT

Zero runtime dependencies is a deliberate result rather than a boast. The Worker runs on the Workers runtime and uses WebCrypto, `fetch` and the D1 binding. The CLI runs on Node 20 and uses `node:*` builtins and the `node:test` runner. A package that a stranger installs globally and points at their own Cloudflare account is a package whose dependency tree is a security surface, and the smallest possible tree is none.

2.2 Composition

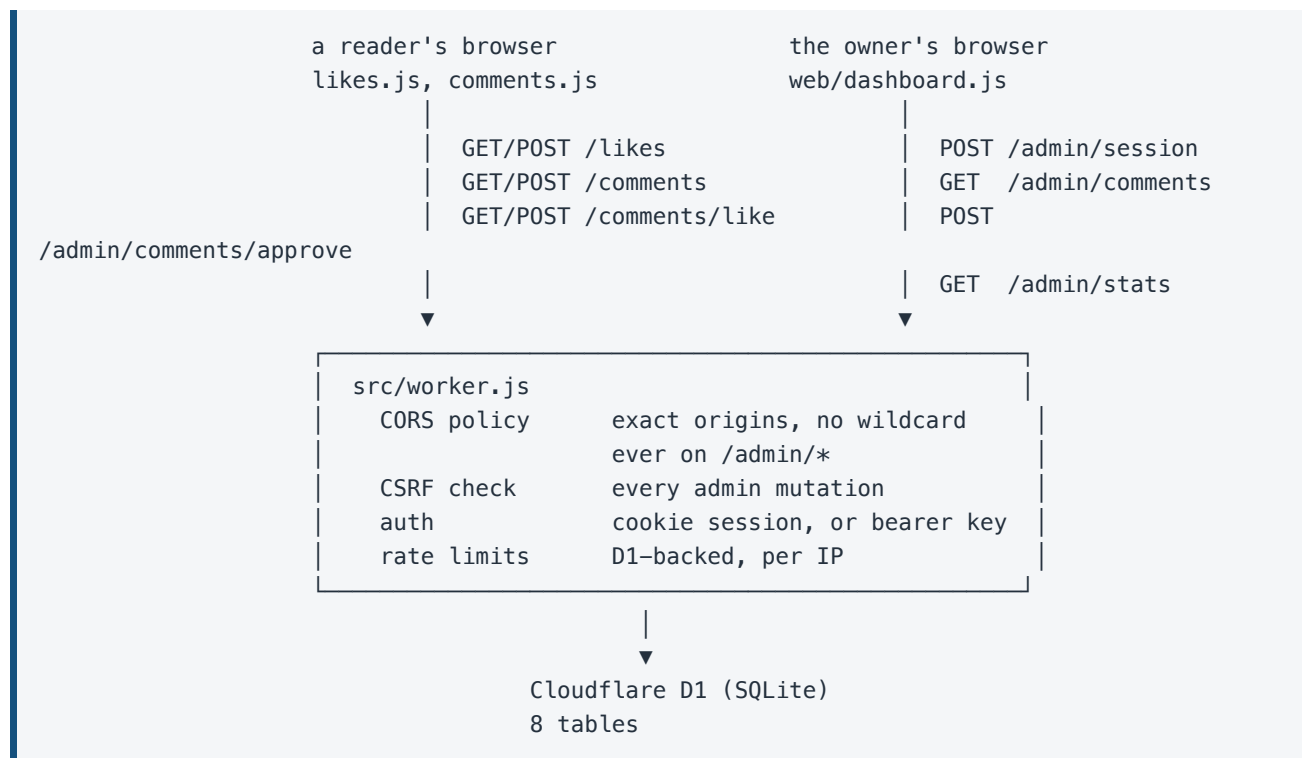
FILE	LINES	JOB
<code>src/worker.js</code>	2,204	The Worker: routing, queries, moderation, stats, audit
<code>web/dashboard.js</code>	1,919	The admin UI behaviour
<code>test/worker-admin-flow.test.mjs</code>	1,129	Fetch-handler tests against a mocked D1 binding
<code>test/worker-security.test.mjs</code>	809	Unit tests for the security helpers
<code>src/setup-env.js</code>	711	The guided first run
<code>src/backout.js</code>	621	Teardown: undo what setup created
<code>src/wrangler-config.js</code>	610	<code>wrangler.toml</code> generation and editing

FILE	LINES	JOB
src/worker-security.mjs	544	CORS, CSRF, sessions, rate limiting, safe errors
src/admin-key.js	543	Key generation, expiry, rotation, clipboard
src/cli.js	515	Moderation and D1 SQL command builders
src/env-config.js	490	.env reading and writing, sensitive-value handling
client/comments.js	429	The embeddable comment thread
src/admin-session.js	323	Session TTL configuration
client/likes.js	171	The embeddable like button

Two numbers in that table are worth pausing on. `src/backout.js` is **621 lines**, which is more than the setup path it undoes in several places, and it exists because a guided setup that creates a database, a Worker, two config files and a secret has created five things a user may want to be rid of, and leaving them behind is how a trial install becomes a bill. **The two Worker test files total 1,938 lines against a 2,204-line Worker**, which is close to parity for the component that has no other way of being checked.

3. The runtime shape

One Worker serves two audiences on one origin.



3.1 The public surface

ENDPOINT	DOES
GET /likes?path=	the like count for a page path
POST /likes?path=	increment it, and record a <code>page_like</code> event
GET /comments?path=	approved, non-hidden comments as a nested thread
POST /comments	create a pending comment or reply
GET /comments/like?commentId=	a comment's like count
POST /comments/like?commentId=	increment it, record a <code>comment_like</code> event

Everything public either reads an aggregate or writes something that is invisible until a human approves it. That is the whole authorization model on this side, and it is why the public surface needs no accounts: **nothing a stranger can do is visible to another stranger without the owner acting.**

3.2 The admin surface

Thirteen endpoints covering session lifecycle, summary and stats, comment listing and the five moderation verbs (approve, reject, hide, unhide, delete), ranked like listings, denied-keyword settings, audit logs, and Worker metadata.

`hide` and `delete` being separate verbs is the small design decision in that list. Hiding keeps a comment approved and stamps `hidden_at`, so it can come back; deleting removes it and its child replies permanently. A moderation UI with only one destructive verb encourages using it for both cases.

4. Security architecture

This is the part of the project worth reading. Every control here exists because a static-site engagement layer is a specific and slightly awkward threat model: an API deliberately called cross-origin from a site the API does not host, and an admin console that is itself a static file on a different origin again.

4.1 CORS, and the one rule that cannot bend

Allowed origins are exact strings. The CLI refuses to store `*` and the Worker never honours a wildcard on `/admin/*`.

The reason those are two separate defences is that they fail differently. A CLI that refuses the wildcard protects the person setting it up from a mistake; a Worker that refuses to honour one protects the deployment from a config that arrived some other way, such as being edited in the Cloudflare dashboard. **A control implemented only in the tool that writes the config is a control that a hand-edit removes.**

The CLI also strips wildcard entries when a real origin is added, so an install that started permissive during development does not stay that way by accident.

4.2 Sessions without a session server

The admin key is accepted exactly once, at `POST /admin/session`, and exchanged for a cookie:

- `'__Host-'` **prefixed**, which the browser enforces: it requires `Secure`, requires `Path=/`, and forbids a `Domain` attribute, so the cookie cannot be set by a subdomain or scoped to one.
- `'HttpOnly'`, so dashboard JavaScript cannot read it and an XSS in the dashboard cannot exfiltrate it.
- **Signed**, with `ADMIN_SESSION_SECRET` where set, falling back to `ADMIN_API_KEY`. The separate secret is the better configuration because it decouples session validity from key rotation.
- **Carrying a 'jti'**, recorded in the `admin_sessions` table, which is what makes an individual session revocable. A signed stateless token cannot be revoked; a signed token with a row can.

The fallback deserves a note. Falling back to the admin key means an install that never sets a session secret still gets signed sessions, and rotating the key invalidates every session, which is the safe direction for a fallback to fail in.

4.3 CSRF on an API that is cross-origin by design

The usual same-origin defence is unavailable here: the dashboard is on the owner's site and the Worker is on `workers.dev` or a route, so admin calls are cross-origin by construction. Cookies plus cross-origin requests is the classic CSRF setup.

The answer is an **Origin check on every admin mutation**, allowing exactly the configured origins and rejecting a missing Origin rather than treating absence as trust. The test suite covers all three cases:

allowed, denied, and missing. A cross-origin `text/plain` POST, which is the shape that historically slipped past `Content-Type`-based defences, is asserted to return **403**.

4.4 Rate limiting with no shared memory

Workers are isolates. There is no process to hold a counter, and an in-memory limiter is per-isolate, which means per-region and per-restart.

So the counters live in D1, in two tables built the same way:

```
CREATE TABLE IF NOT EXISTS auth_attempts (
  bucket_key    TEXT NOT NULL,
  window_start  INTEGER NOT NULL,
  attempt_count INTEGER NOT NULL DEFAULT 0,
  lockout_until INTEGER,
  UNIQUE (bucket_key, window_start)
);
```

One row per bucket per window, upserted with `INSERT ... ON CONFLICT`, so a concurrent burst across isolates increments one row rather than racing several. The in-memory limiter is kept and mirrors it, which makes the common case cheap without making the guarantee depend on it.

The configured limits:

SURFACE	LIMIT
Failed admin logins	5 per 15 minutes per IP
Comment creation	5 per minute per IP
Page likes	30 per minute per IP
Comment likes	30 per minute per IP

Only failures count on the login limiter. A limiter that counts successes locks out the legitimate owner during a busy moderation session, which is the failure mode that gets rate limiting removed.

Client IP comes from `CF-Connecting-IP` **only**. `X-Forwarded-For` is caller-supplied and therefore not an identifier; a limiter keyed on a header the attacker writes is not a limiter. The audit log records the same field for the same reason.

4.5 Errors that say nothing and can still be traced

`buildSafeErrorResponse` returns a sanitised 500 carrying a correlation id, with a base64url fallback id when one cannot be derived. The response tells the caller nothing about the failure; the id lets the owner find it in the logs.

That pairing is the whole point. An error handler that returns the stack is a disclosure; an error handler that returns nothing at all makes the owner's own bug reports unusable.

4.6 Email at rest

Commenter emails are optional and, when `DATA_ENCRYPTION_KEY` is set, encrypted with **AES-GCM** before they reach the database, with a per-record IV.

Opt-in rather than mandatory is a defensible position for a field that is itself optional, and the honest framing is that an install without the key stores emails in plaintext in the owner's own D1. The documentation says so rather than implying the encryption is always on.

4.7 What the tests actually assert

The Worker's fetch handler is driven against a mocked D1 binding, which is what makes the following checkable without a Cloudflare account:

- The full admin session POST, GET and DELETE lifecycle.
- The 401 gate on `/admin/*`.
- Audit-log `INSERT` assertions, so a moderation action that is not logged fails a test.
- `/admin/worker` never leaking `ADMIN_API_KEY` or `ADMIN_SESSION_SECRET`.
- A 403 for a cross-origin `text/plain` POST.
- 429 with `Retry-After` after five failed admin keys.
- The rate limiter counting failures only.
- `getClientIp` reading `CF-Connecting-IP` only.
- Session token sign, verify, forgery and expiry, and every cookie flag.

That list is the security policy in executable form, which is the only form of it that cannot drift from the code.

5. The data model

Eight tables and nine indexes. The two tables that are not obvious are the interesting ones.

TABLE	HOLDS
<code>post_likes</code>	one row per path, with a <code>CHECK (count >= 0)</code>
<code>post_comments</code>	the thread, with <code>parent_id</code> , a status enum and <code>hidden_at</code>
<code>comment_denied_keywords</code>	keywords that auto-reject a comment
<code>engagement_events</code>	one row per like or comment, for the stats graph
<code>admin_audit_logs</code>	every protected action, with a key fingerprint and IP
<code>auth_attempts</code>	the login rate limiter
<code>public_rate_limits</code>	the public-endpoint rate limiter
<code>admin_sessions</code>	one row per issued session, so it can be revoked

Three schema decisions are load-bearing.

Constraints are in the schema, not in the Worker. `CHECK (count >= 0)` and `CHECK (status IN ('pending', 'approved', 'rejected'))` mean a bug in the Worker cannot leave a negative like count or an unrecognised status in the database. A validation that exists only in application code is a validation that the next code path can skip.

`engagement\_events` is separate from the counters. The counters are what a page reads and must be a single-row lookup; the events are what the dashboard graph aggregates over a range. Deriving the counter from the events on every page view would make the cheapest read in the system the most expensive one.

`admin\_sessions` exists purely to make a stateless token stateful. A signed cookie needs no table, and that is precisely the problem: it cannot be revoked before it expires. One row per session is the price of a logout that actually logs out.

Every table is created with `IF NOT EXISTS`, and the Worker also runs idempotent `ensure*` DDL for the two rate-limit tables, so an install that predates them acquires them without the owner running a migration. **That is a deliberate answer to the migration problem for a package deployed onto other people's infrastructure:** there is no migration runner, so every schema change has to be additive and self-installing.

6. Setup, and the parts that refuse

The CLI is 30 commands, and the design principle running through them is that the destructive and the disclosing ones ask first.

`urthreads setup-env` can authenticate Wrangler, create or reuse a D1 database, write `.env` with `0600` permissions, create `wrangler.toml`, initialise the schema, and deploy the Worker. What it will not do:

It will not silently overwrite `wrangler.toml`. If one exists, it asks again before replacing it, and the default answer is no. It then names the specific consequence of keeping the existing file, which is that `.env` and `wrangler.toml` may point at different databases, and prints the two commands that reconcile them.

It will not print secrets by default. `env` and `wrangler` listing and getting hide sensitive values unless `-show-sensitive` is passed, and `wrangler set` refuses to store sensitive keys in `wrangler.toml` at all, because that file is commonly committed.

It will not let production keep a localhost origin quietly. A warning is emitted when production origins contain localhost.

`urthreads backout` and the three `clean` commands undo it. Deleting the deployed Worker requires an explicit confirmation.

Admin keys can expire. `urthreads admin-key --expires 30d` generates a key, copies it to the clipboard, optionally stores it as the Worker secret, and can update `ADMIN_API_KEY_EXPIRES_AT` in `wrangler.toml` and prompt for a deploy. A finite key expiry on a single-key admin surface is the cheapest available limit on the damage from a leaked key.

7. The dashboard

Three static files: `index.html`, `dashboard.js`, `styles.css`. Built into the owner's site output with `urthreads dashboard set ./public urthreads` and `urthreads dashboard build`, typically served at `/urthreads/`.

Two client-side controls matter more than they look:

It refuses a plain-http worker origin, except for `localhost`, `127.0.0.1` and `:::1` on any port. The check happens *before* the admin key is sent, which is the only placement that helps: a check after the request has already leaked the key.

It ships a strict Content-Security-Policy and a no-referrer policy, with `connect-src` allowing `https:` worker origins plus the localhost development origins. A dashboard that renders untrusted comment text is a dashboard that has to assume it will one day render something hostile, and CSP is the control that survives a rendering bug.

The origin the dashboard is hosted on must be added to `ALLOWED_ORIGINS`, scheme plus host plus optional port, not including the path. That is stated in the documentation because getting it wrong produces a CORS failure that looks like a broken deployment.

8. What this project demonstrates

It is worth being explicit, because the temptation with a portfolio is to lead with the largest thing rather than the most legible one.

Security engineering on an ordinary threat model. Sections 4 and 5 are not AI work and not novel work. They are the standard controls, applied correctly, in a setting where each one had a specific reason to be difficult: sessions with no session server, CSRF where cross-origin is the design, rate limiting with no shared memory, and schema evolution with no migration runner.

Tests that assert the security policy. The list in Section 4.7 is the whole of the reason to believe any of Section 4. Every claim there has a test, and the two Worker test files are close to line parity with the Worker.

Defaults as policy. The wildcard refused in two places, sensitive values hidden by default, the http origin refused before the key is sent, the confirmation before deleting a Worker. Every one of these is a decision that the person deploying will never make, so the software makes it.

Shipped, and used. It is on npm at 1.2.2, it runs the engagement layer on yigityildiz.dev, and it is the one project here where somebody other than the author has to be able to install it from a README.

9. Open problems

9.1 The dashboard is not tested. 1,919 lines of `web/dashboard.js` have no test file. The Worker is well covered and the CLI is well covered; the third surface is not, and it is the one that renders untrusted text.

9.2 One admin key, one role. There is no notion of a second moderator, and no scopes. For the intended deployment, a personal site, that is correct. It is the first thing that breaks if anyone else ever wants to use it for a team blog.

9.3 Email encryption is opt-in. Section 4.6. The safer default would be to generate a key during setup rather than leaving the field plaintext when the variable is unset.

9.4 No spam defence beyond keywords and rate limits. `comment_denied_keywords` and 5 comments per minute per IP are the whole of it. That is enough for a personal site and would not survive being targeted.

9.5 Rate limit rows are never pruned by anything but ``pruneExpiredAdminSessions`` sibling paths. Old windows accumulate. The volume is small and the tables are indexed, so this is a housekeeping item rather than a defect, but it is unbounded growth in a database the owner pays for.

10. Conclusion

urthreads is a small system that solves a real problem for a specific person, and the interesting engineering in it is entirely in the constraints rather than the features.

The constraint that shaped it is that the software is deployed by someone who is not an operator, onto infrastructure they own, with no way for the author to fix a bad configuration afterwards. Everything in Section 4 and Section 6 follows from that: the wildcard refused in two places rather than one, the confirmation before the destructive command, the secrets hidden unless asked for, the schema that installs its own additions because there is nobody to run a migration.

The measure that matters is Section 4.7. Twelve security properties, each with a test that fails if the property stops holding. That is the difference between a system that is secure and a system whose security anyone can check, and it is the same standard the other documents in this portfolio are written to.

Appendix A: Verification log

Measured against the working tree at commit [124e24a](#) on 8 September 2026.

A.1 Tests

```
$ npm test
i tests 229
i suites 0
i pass 229
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 443.930375
EXIT=0
```

Distribution across the 14 files:

FILE	TESTS
worker-security.test.mjs	63
worker-admin-flow.test.mjs	33
setup-env.test.js	21
cli.test.js	17
env-config.test.js	14
wrangler-config.test.js	14
backout.test.js	14
admin-key.test.js	11
urthreads.test.js	10
setup-dev.test.js	9
dashboard-config.test.js	8
admin-session.test.js	6
help-format.test.js	5
example-config.test.js	4

96 of 229 tests, 42%, are on the Worker, which is 11% of the source. That is the intended imbalance: the Worker is the only part that is exposed to the internet.

A.2 Scale

```
$ git ls-files '*.js' '*.mjs' '*.ts' '*.sql' '*.css' '*.html' \
  | grep -v node_modules | xargs wc -l | tail -1
19605 total

$ git rev-list --count HEAD
35

$ git log --reverse --format='%ad' --date=short | head -1
2026-05-19

$ git log -1 --format='%h %ad %s' --date=short
124e24a 2026-09-03 chore: ignore graphify build output
```

A.3 Dependencies

```
$ node -e "const p=require('./package.json');
           console.log(Object.keys(p.dependencies||{}).length,
                        Object.keys(p.devDependencies||{}).length)"
0 1
```

A.4 Surface

```
$ grep -oE "'/(admin/)?[a-z-]+(/[a-z-]+)*'" src/worker.js | sort -u | wc -l
20

$ grep -cE '^export (function|const|async function)' src/worker-security.mjs
25

$ grep -c 'CREATE TABLE IF NOT EXISTS' src/schema.sql
8

$ grep -c 'CREATE INDEX IF NOT EXISTS' src/schema.sql
9
```

A.5 Rate-limit constants

From `src/worker.js`, lines 37 to 42:

```
const ADMIN_LOGIN_MAX_FAILURES = 5;
const ADMIN_LOGIN_WINDOW_MS = 15 * 60 * 1000;
const PUBLIC_RATE_LIMIT_LIKES = 30;
const PUBLIC_RATE_LIMIT_COMMENT_LIKES = 30;
const PUBLIC_RATE_LIMIT_COMMENTS = 5;
const PUBLIC_RATE_LIMIT_WINDOW_MS = 60_000;
```

Prepared 8 September 2026 against the working tree at commit `124e24a`, npm release 1.2.2. All commands in Appendix A were executed during preparation and their output is reproduced verbatim.